

SIMATS ENGINEERING

ASSESSMENT TOOL 3: Modelling Comparative Analysis Assignment

**COURSE NAME: ARTIFICIAL INTELLIGENCE COURSE CODE: MLA01
AND EXPERT SYSTEMS**

COURSE OUTCOME COVERED

CO 5: Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100
No. of Questions: 1

Time: 90 Mins

Annexure A Modelling Comparative Analysis Assignment

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Criterion	Weightage (%)	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Domain Knowledge and Fact Identification	10%	
3. Production Rule Modelling	15%	
4. Propositional Logic Modelling	10%	
5. First-Order Logic Modelling	15%	
6. Prolog Modelling and Implementation	15%	
7. Forward and Backward Chaining Demonstration	10%	
8. Comparative Analysis of Modelling Approaches	10%	
9. Results, Discussion and Model Selection	3%	
10. Documentation, GitHub Repository and Presentation	2%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

S.No.	Question / Problem Statement
1	Automobile Fault Diagnosis: An automobile service center wants to identify vehicle faults based on symptoms such as engine overheating, starting failure, abnormal noise, low mileage, and warning indicators. Model the problem using production rules, propositional logic, first-order logic, and Prolog. Demonstrate forward and backward chaining and compare the approaches based on expressiveness, inference, complexity, and suitability.
2	Industrial Machine Fault Diagnosis: A manufacturing plant needs to identify machine faults using observations such as abnormal vibration, high temperature, unusual noise, pressure variation, and reduced production speed. Develop different knowledge representation models and compare their ability to represent machine components, relationships, rules, and diagnostic conclusions.
3	Cybersecurity Threat Detection: A security operations center wants to identify threats from repeated login failures, unusual login locations, privilege escalation, suspicious file access, and abnormal network traffic. Model the threat-detection problem using production rules, propositional logic, first-order logic, and Prolog, and compare forward and backward reasoning for deriving security conclusions.
4	Healthcare Diagnosis: A healthcare system needs to identify possible diseases from symptoms such as fever, cough, fatigue, breathing difficulty, and body pain. Construct comparative knowledge models using production rules, propositional logic, first-order logic, and Prolog. Analyze which representation provides better expressiveness and reasoning capability for diagnosis.
5	Agricultural Crop Disease Diagnosis: Farmers need assistance in identifying crop diseases based on leaf color, spots, wilting, soil conditions, humidity, and pest observations. Represent the agricultural knowledge using different modelling approaches and compare their effectiveness in representing relationships, applying rules, and deriving recommendations.
6	Banking Loan Eligibility: A bank wants to determine loan eligibility using income, credit score, employment status, existing loans, and repayment history. Model the decision-making process using production rules, propositional logic, first-order logic, and Prolog, and compare the reasoning capabilities of each approach.

Structure of the Report:

S.No.	Report Section	Expected Content
1	Problem Statement & Objectives	Describe the selected real-world problem, domain, decision-making requirement, and objectives of the modelling exercise.
2	Domain Knowledge & Facts	Identify entities, attributes, facts, relationships, conditions, and conclusions relevant to the selected problem.
3	Production Rule Model	Represent the domain using IF–THEN production rules and explain how the rules derive conclusions.
4	Propositional Logic Model	Represent the same problem using propositions, logical connectives, and suitable inference statements.
5	First-Order Logic Model	Represent entities, relationships, properties, and rules using predicates and quantifiers.
6	Prolog Model & Implementation	Convert the knowledge into Prolog facts, rules, predicates, and queries. Include relevant code and outputs.
7	Forward & Backward Chaining	Demonstrate both reasoning mechanisms using selected facts/goals and provide the inference sequence.
8	Comparative Analysis (Table structure given)	Compare the models based on expressiveness, inference mechanism, complexity, scalability, flexibility, explainability, and suitability.
9	Results, Discussion & Recommended Model	Analyze the results and justify which approach is most appropriate for the selected real-world problem. Discuss limitations and possible improvements.
10	Conclusion, References & GitHub	Summarize findings, provide references, and include the GitHub repository link containing Prolog code, models, test cases, and supporting materials.

Compulsory Comparative Table:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge Representation				
Expressiveness				
Rule Representation				
Inference Mechanism				
Forward Chaining				
Backward Chaining				
Handling Relationships				
Scalability				
Explainability				
Ease of Implementation				
Suitability for the Problem				

**CYBERSECURITY THREAT DETECTION USING
KNOWLEDGE REPRESENTATION AND REASONING**

CO5 ASSESSMENT TOOL -3

MODELLING COMPARATIVE ANALYSIS ASSIGNMENT

**Submitted in the partial fulfilment for the Course of
MLA0104-ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS**

to the award of the degree of

BACHELOR OF TECHNOLOGY

IN

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

RASHMITHA SENTHIL KUMAR (192525034)



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

AUGUST 2026

1. PROBLEM STATEMENT & OBJECTIVES:

Cybersecurity systems receive large numbers of login, file-access and network events. Manually examining every event is difficult and may delay threat detection. This report develops an expert system that identifies cybersecurity threats using repeated login failures, unusual login locations, privilege escalation, suspicious file access and abnormal network traffic.

The objectives are to:

- Represent cybersecurity knowledge using four modelling approaches.
- Develop accurate IF–THEN production rules.
- represent the problem using propositional logic and first-order logic.
- Implement the knowledge using Prolog facts, rules and queries.
- Demonstrate forward and backward chaining.
- Compare the models and select the most suitable approach.

Cybersecurity Expert System Architecture

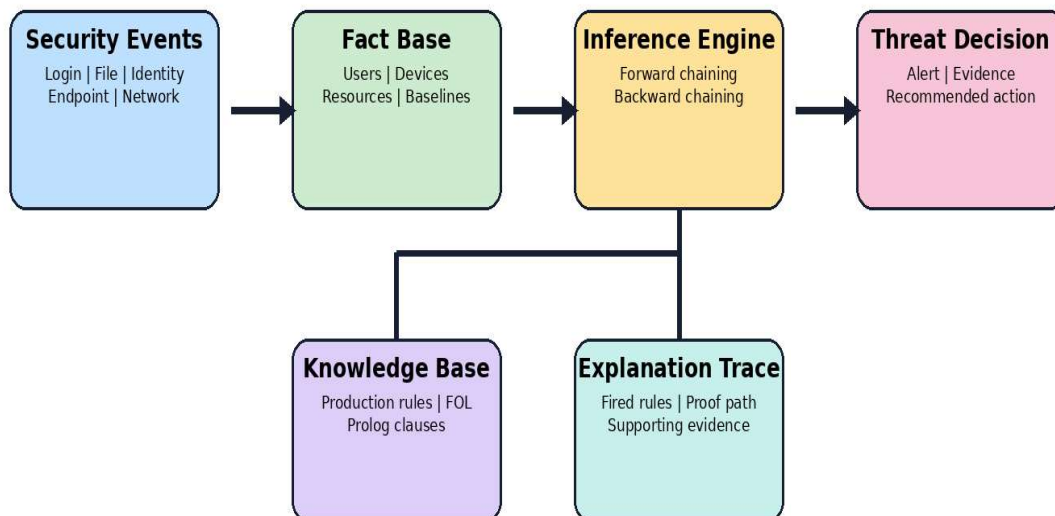


Figure 1: Architecture of the Cybersecurity Threat Detection Expert System

2. DOMAIN KNOWLEDGE & FACTS:

The main entities are users, devices, locations, files, network events and security alerts. Important attributes include username, device name, login-failure count, login location, privilege level, accessed file and network-traffic condition.

The case considers the following facts:

- Alice has six failed login attempts.
- Alice logged in from an unusual location.
- Alice performed privilege escalation.
- Alice accessed a sensitive payroll file.
- Alice's workstation produced abnormal network traffic.
- Workstation ws17 belongs to Alice.

A repeated-login condition occurs when the failure count is five or more. Combining several suspicious conditions may produce conclusions such as brute-force attempt, compromised account, possible data exfiltration and critical threat.

3. PRODUCTION RULE MODEL:

The domain is represented using the following IF-THEN rules:

1. IF failed login count is five or more, THEN repeated login failures are detected.
2. IF repeated login failures are detected, THEN infer a brute-force attempt.
3. IF a user logs in from an unusual location, THEN infer a suspicious login.
4. IF a brute-force attempt and suspicious login occur for the same user, THEN infer a compromised account.
5. IF privilege escalation and suspicious file access occur, THEN infer privilege misuse.
6. IF a user accesses a sensitive file and the user's device has abnormal traffic, THEN infer possible data exfiltration.
7. IF an account is compromised and privilege escalation occurs, THEN classify the account as high risk.

8. IF a high-risk account also shows data exfiltration, THEN infer a critical threat.
9. IF a critical threat is detected, THEN recommend isolating the device.
10. IF a critical threat is detected, THEN recommend resetting the user's credentials.

These rules derive security conclusions by combining related evidence belonging to the same user and device.

4. PROPOSITIONAL LOGIC MODEL:

Let:

- L: Alice has at least five failed logins.
- U: Alice logged in from an unusual location.
- P: Alice performed privilege escalation.
- S: Alice accessed a sensitive file.
- N: Alice's device generated abnormal traffic.
- B: A brute-force attempt is detected.
- A: Alice's account is compromised.
- E: Possible data exfiltration is detected.
- C: A critical threat is detected.

Logical statements:

$$\begin{aligned}L &\rightarrow B \\(B \wedge U) &\rightarrow A \\(S \wedge N) &\rightarrow E \\(A \wedge P \wedge E) &\rightarrow C\end{aligned}$$

Because L, U, P, S and N are true, modus ponens derives B, followed by A, E and finally C.

Propositional logic is clear for a small case but requires separate propositions for every user and device.

5. FIRST-ORDER LOGIC MODEL:

Predicates:

- FailedCount(u,n): User u has n failed logins.
- UnusualLocation(u,l): Location l is unusual for user u.
- Escalated(u): User u performed privilege escalation.
- SuspiciousAccess(u,r): User u accessed resource r suspiciously.
- Owns(u,d): User u owns device d.
- AbnormalTraffic(d): Device d has abnormal traffic.
- Compromised(u): The account of user u is compromised.
- CriticalThreat(u): User u is associated with a critical threat.

FOL rules:

$$\forall u, n [FailedCount(u, n) \wedge n \geq 5 \rightarrow BruteForce(u)]$$
$$\forall u, l [BruteForce(u) \wedge UnusualLocation(u, l) \rightarrow Compromised(u)]$$
$$\forall u, d, r [Owns(u, d) \wedge SuspiciousAccess(u, r) \wedge AbnormalTraffic(d) \rightarrow Exfiltration(u)]$$
$$\forall u [Compromised(u) \wedge Escalated(u) \wedge Exfiltration(u) \rightarrow CriticalThreat(u)]$$

First-order logic represents users, devices, resources and their relationships using reusable variables and quantifiers.

6. PROLOG MODEL & IMPLEMENTATION:

failed_login_count(alice, 6).

unusual_location(alice, berlin).

privilege_escalation(alice).

suspicious_file_access(alice, payroll_db).

abnormal_network_traffic(ws17).

owns(alice, ws17).

repeated_login_failures(User) :-

failed_login_count(User, Count),

Count >= 5.

brute_force_attempt(User) :-

repeated_login_failures(User).

suspicious_login(User) :-

unusual_location(User, _).

compromised_account(User) :-

brute_force_attempt(User),

suspicious_login(User).

possible_data_exfiltration(User) :-

owns(User, Device),

suspicious_file_access(User, _),

abnormal_network_traffic(Device).

critical_threat(User) :-

compromised_account(User),

privilege_escalation(User),

possible_data_exfiltration(User).

recommend(User, isolate_device) :-

critical_threat(User).

recommend(User, reset_credentials) :-

critical_threat(User).

Queries and outputs:

?- repeated_login_failures(alice).

true.

?- compromised_account(alice).

true.

?- possible_data_exfiltration(alice).

true.

?- critical_threat(alice).

true.

?- recommend(alice, Action).

Action = isolate_device ;

Action = reset_credentials.

?- critical_threat(bob).

false.

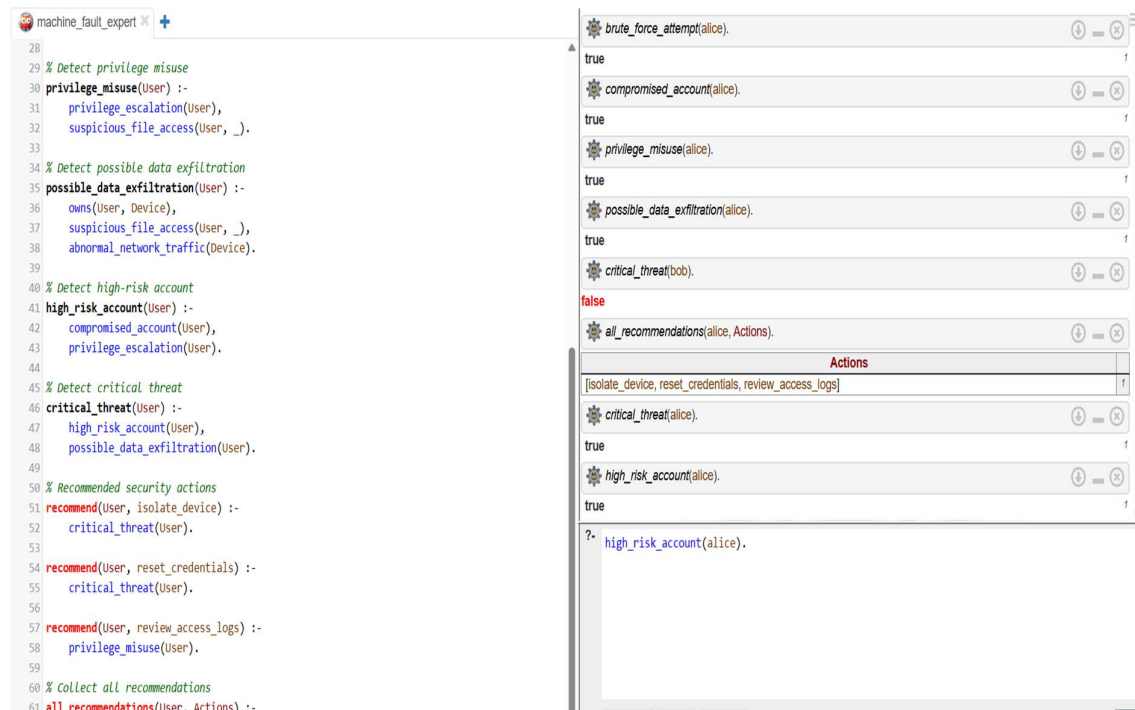


Figure 2: Real Validation Output of the Implemented Cybersecurity Expert System

In SWISH, enter only the query in the query box. Do not type ?- inside the query and do not attempt to consult the program using [filename]., because SWISH automatically runs the code written in its editor.

7. FORWARD & BACKWARD CHAINING:

Forward chaining begins with the available facts:

1. Six failed logins derive `repeated_login_failures(alice)`.
2. Repeated failures derive `brute_force_attempt(alice)`.
3. The unusual location derives `suspicious_login(alice)`.
4. Brute force and suspicious login derive `compromised_account(alice)`.
5. Suspicious file access and abnormal traffic from Alice's device derive `possible_data_exfiltration(alice)`.
6. Account compromise, privilege escalation and exfiltration derive `critical_threat(alice)`.

Backward chaining begins with the goal:

`critical_threat(alice)`

Prolog checks whether Alice's account is compromised, whether she performed privilege escalation and whether data exfiltration is possible. Each subgoal is matched with the available facts and rules. Since all subgoals succeed, the final goal is proved.

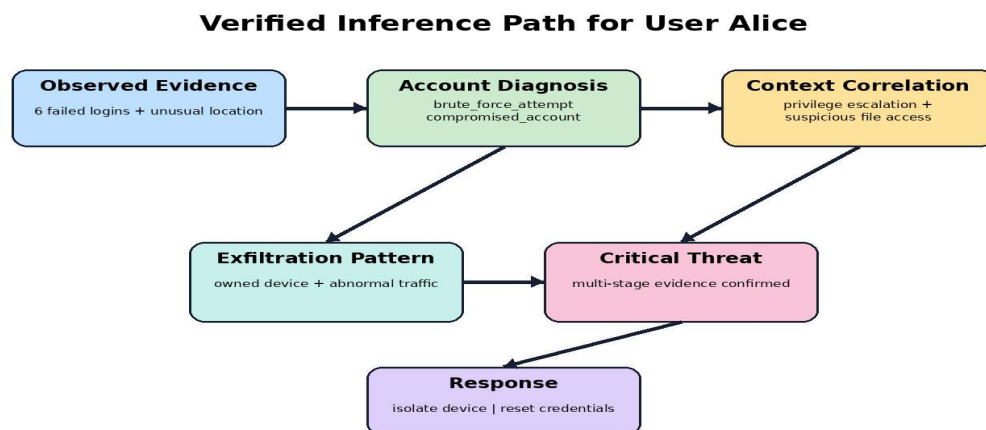


Figure 3: Forward and Backward Inference Path for Cybersecurity Threat Detection

8. COMPARATIVE ANALYSIS:

Parameter	Production Rules	Propositional Logic	First-Order Logic	Prolog
Knowledge representation	IF-THEN rules	Symbols and formulae	Predicates and quantifiers	Facts and rules
Expressiveness	Moderate	Limited	Very high	High
Rule representation	Condition-action	Logical implication	Quantified formulae	Horn clauses
Inference mechanism	Rule firing	Modus ponens	Unification and resolution	SLD resolution
Forward chaining	Natural	Possible	Possible	Must be programmed
Backward chaining	Possible	Proof-based	Theorem proving	Native
Relationships	Moderate	Weak	Excellent	Excellent
Scalability	Good with rule engines	Poor for many entities	Powerful but complex	Good for bounded systems
Explainability	Very high	High for small models	High but complex	High
Ease of implementation	Easy	Easy for small cases	Difficult	Easy to moderate
Suitability	Good for alert generation	Suitable as a basic model	Best formal representation	Best executable prototype

Production rules are suitable for continuous alert generation. FOL represents complex relationships accurately, while Prolog provides a practical executable system with understandable reasoning.

9. RESULTS, DISCUSSION & RECOMMENDED MODEL:

The system successfully detected repeated login failures, account compromise, possible data exfiltration and a critical cybersecurity threat for Alice. The unsupported query for Bob returned false, showing that conclusions are generated only when relevant evidence exists.

Prolog is the recommended model because it supports facts, reusable variables, relationships, rules and direct queries. It also performs backward chaining automatically and produces explainable conclusions. However, the model uses fixed thresholds and a small test dataset. It can be improved by adding timestamps, adaptive thresholds, confidence scores and real security-event data.

10. CONCLUSION,:

The comparative analysis shows that each approach has a different advantage. Production rules provide clear operational decisions, propositional logic supports simple logical verification, FOL represents complex relationships and Prolog converts the knowledge into an executable expert system. Prolog is therefore the most suitable model for this cybersecurity threat-detection prototype.

11.REFERENCES:

1. Bratko, Ivan. Prolog Programming for Artificial Intelligence. 4th ed. Addison-Wesley, 2012.
2. National Institute of Standards and Technology. The NIST Cybersecurity Framework 2.0. 2024.
3. Russell, Stuart J., and Peter Norvig. Artificial Intelligence: A Modern Approach. 4th ed. Pearson, 2021.
4. SWI-Prolog. "SWI-Prolog Reference Manual." Accessed August 2026.
<https://www.swi-prolog.org/pldoc/man?section=manual>.
5. MITRE. "MITRE ATT&CK Enterprise Matrix." Accessed August 2026.
<https://attack.mitre.org/>.

12.GITHUB REPOSITORY:

https://github.com/Rashmitha22/MLA0104_AIES/tree/main/ASSESSMENT%20TOOL/C05/AT3

EVALUATION RUBRICS

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, constraints, facts, and expected outcomes.	7–8% – Good understanding with minor omissions.	5–6% – Basic understanding with limited analysis.	0–4% – Problem is unclear or poorly understood.
2. Domain Knowledge and Fact Identification	10%	9–10% – Identifies comprehensive and relevant facts, entities, relationships, conditions, and domain knowledge.	7–8% – Identifies most relevant knowledge with minor gaps.	5–6% – Basic facts and entities are identified.	0–4% – Important domain knowledge is missing or incorrect.
3. Production Rule Modelling	15%	13–15% – Develops accurate, complete, consistent IF–THEN production rules that effectively represent the problem.	10–12% – Rules are appropriate with minor errors or omissions.	7–9% – Basic rules are developed but several are incomplete or unclear.	0–6% – Rules are largely incorrect, incomplete, or inappropriate.
4. Propositional Logic Modelling	10%	9–10% – Correctly represents domain knowledge using propositions, logical connectives, and valid inference statements.	7–8% – Good propositional representation with minor errors.	5–6% – Basic representation with limited logical formulation.	0–4% – Representation is incorrect or incomplete.
5. First-Order Logic Modelling	15%	13–15% – Accurately models entities, relationships, properties, variables, predicates, and quantifiers with valid FOL statements.	10–12% – Good FOL model with minor errors or omissions.	7–9% – Basic FOL representation with limited relationships or incorrect usage of some constructs.	0–6% – FOL model is largely incorrect or missing.

6. Prolog Modelling and Implementation	15%	13–15% – Successfully converts the knowledge model into Prolog facts, rules, predicates, and queries with correct outputs.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works but has limited functionality.	0–6% – Implementation is incomplete or non-functional.
7. Forward and Backward Chaining Demonstration	10%	9–10% – Clearly demonstrates both forward and backward chaining with correct inference sequences and conclusions.	7–8% – Both mechanisms are demonstrated with minor gaps.	5–6% – Basic demonstration with limited reasoning explanation.	0–4% – Chaining is absent or incorrectly demonstrated.
8. Comparative Analysis of Modelling Approaches	10%	9–10% – Provides a detailed and insightful comparison based on expressiveness, inference, complexity, scalability, flexibility, and explainability.	7–8% – Provides a meaningful comparison using appropriate parameters.	5–6% – Basic comparison with limited criteria or analysis.	0–4% – Little or no meaningful comparison.
9. Results, Discussion and Model Selection	3%	3% – Clearly analyzes results and convincingly justifies the most suitable approach for the selected problem.	2% – Provides good analysis and reasonable justification.	1% – Basic discussion with limited justification.	0% – No meaningful analysis or justification.
10. Documentation, GitHub Repository and Presentation	2%	2% – Report is complete and well structured, GitHub repository is organized, and presentation is professional.	1.5% – Good documentation and presentation with minor issues.	1% – Basic documentation with some omissions.	0% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				